

A Project Report on

Smart Labs Zonal Control Using Machine Learning

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

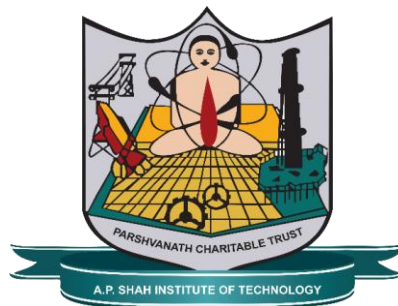
By

Karan Vethody: 22106023

Vedant Vethekar: 22106090

Jay Patil: 22106020

Under the Guidance of
Prof. Shraddha Dalvi

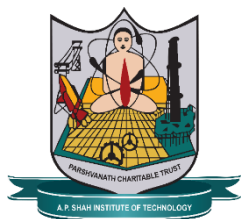


Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026



A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled “Smart Labs Zonal Control Using Machine Learning” is a bonafide work of **Karan Vethody (22106023), Vedant Vethekar (22106090) , Jay Patil (22106020)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Science & Engineering (Artificial Intelligence & Machine Learning)**.

Prof. Shraddha Dalvi
Guide

Prof. Sayali Bhadan
Project Co-ordinator

Dr. Jaya Gupta
Head of Department

Dr. Uttam Kolekar
Principal



A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled “*Smart Lab Zonal Control Using Machine Learning*” by *Karan Vethody, Jay Patil, Vedant Vethekar* is approved for the degree of *Bachelor of Engineering in Computer Science & Engineering (Artificial Intelligence & Machine Learning)*, 2025-26.

Examiner Name

Signature

1. _____

2. _____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Karan Vethody 22106023

Vedant Vethekar 22106090

Jay Patil 22106020

Date:

Abstract

Modern university laboratories rely heavily on digital infrastructure, leading to high energy consumption and significant wastage due to irregular occupancy patterns. Conventional automation techniques such as fixed scheduling and basic motion sensors attempt to reduce this wastage; however, they fail to adapt to real-time usage. Fixed schedules operate without considering actual occupancy, while motion sensors often fail to detect stationary users and lack spatial awareness, resulting in inefficient energy utilization. This paper proposes a Smart Lab Automation System that integrates Internet of Things (IoT) hardware, Machine Learning, and Computer Vision to enable intelligent, zone-wise lighting control. The system is built on a Raspberry Pi 5, which functions as an edge computing device for real-time processing. A camera module captures live video streams, which are analyzed using a YOLO-based deep learning model to accurately detect human presence. The laboratory is divided into predefined zones, and lighting is dynamically controlled through GPIO-connected relay modules, ensuring that only occupied areas are illuminated. A web-based monitoring and control interface developed using Flask and Socket.IO enables users to observe system status and manually override automated decisions when necessary. Experimental results demonstrate energy savings ranging from 35% to 70%, along with improved response time and operational accuracy compared to traditional systems. The proposed solution is scalable, cost-effective, and suitable for deployment in smart infrastructure environments. Furthermore, this work aligns with Sustainable Development Goals, specifically SDG 7 (Affordable and Clean Energy), SDG 9 (Industry, Innovation and Infrastructure), and SDG 11 (Sustainable Cities and Communities), by promoting energy-efficient and intelligent automation systems.

Keywords: *Smart Lab Automation, Internet of Things (IoT), Raspberry Pi, YOLO, Computer Vision, Energy Optimization, Zonal Control, Edge Compu*

Contents

1.Introduction	1
1.1 The Need for Energy-Efficient Smart Laboratories.....	1
1.2 Limitations of Existing Automation Systems	2
1.3 Role of Computer Vision and Intelligent Systems	2
1.4 Integration of IoT in Smart Automation	3
1.5 Towards Intelligent Zonal Lighting Control	3
1.6 Proposed Smart Lab Automation System	3
1.7 Vision and Impact	4
2. Survey/ Existing System.....	5
3. Limitation of Existing System.....	11
4. Problem Statement and Objective	13
4.1 Problem Statement	13
4.2 Objectives.....	14
4.3 Scope.....	14
5.Proposed System	16
5.1 Architecture of Proposed System (Detailed Explanation).....	16
5.2 Data Flow Diagram.....	20
5.3 Activity Diagram.....	22
5.4 Key Components of the Proposed Framework.....	24
6. Experimental Setup	26
6.1 Software Requirements.....	26
6.2 Hardware Requirements.....	28
7. Results and Discussion.....	31
8. Conclusion and Future Work.....	39
8.1 Conclusion	39
8.2 Future Work	40
References	42
Appendix	44
Publication.....	62

LIST OF FIGURES

Figure 1: System Dataflow Architecture 17

Figure 2 System Workflow 20

Figure 3 System Design..... 22

Figure 4 Hardware Setup 28

Figure 5 Live Camera feed 35

Figure 6 Zone Definition 35

Figure 7 Light Control Page 36

LIST OF TABLES

Table 1.Research Paper.....9

Table 2.Energy Consumption and Savings Comparison of Conventional and Smart Zonal
Lighting Systems37

ABBREVIATIONS

AI	<i>Artificial Intelligence</i>
ML	<i>Machine Learning</i>
IoT	<i>Internet of Things</i>
ML	<i>Machine Learning</i>
AI	<i>Artificial Intelligence</i>
CV	<i>Computer Vision</i>
YOLO	<i>You Only Look Once</i>
GPIO	<i>General Purpose Input Output</i>
LED	<i>Light Emitting Diode</i>
PIR	<i>Passive Infrared Sensor</i>
API	<i>Application Programming Interface</i>
UI	<i>User Interface</i>
HTTP	<i>HyperText Transfer Protocol</i>
JSON	<i>JavaScript Object Notation</i>
CPU	<i>Central Processing Unit</i>
OS	<i>Operating System</i>
LAN	<i>Local Area Network</i>
TP	<i>True Positive</i>
FP	<i>False Positive</i>
FN	<i>False Negative</i>
SDG	<i>Sustainable Development Goals</i>
Wh	<i>Watt hour</i>

Chapter 1

Introduction

1. Introduction

1.1 The Need for Energy-Efficient Smart Laboratories

In the modern era, educational institutions are increasingly dependent on digital infrastructure, particularly in computer laboratories that operate continuously throughout the day. These laboratories consume a significant amount of electrical energy due to lighting systems, computing equipment, and other electronic devices. However, a major portion of this energy is wasted due to inefficient utilization, especially when lights and systems remain active in the absence of users. Irregular occupancy patterns, varying schedules, and shared usage of laboratory spaces further contribute to unnecessary energy consumption. Despite advancements in automation, many institutions still rely on traditional approaches that do not effectively address this issue.

Conventional automation techniques, such as fixed scheduling and motion sensor-based systems, are commonly used to reduce energy wastage. Fixed scheduling operates on predefined timings without considering actual usage, resulting in lights being turned on even when the laboratory is empty. On the other hand, motion sensors such as Passive Infrared (PIR) switching of lights. These limitations highlight the need for a more intelligent and adaptive system that can respond accurately to real-time occupancy conditions.

1.2 Limitations of Existing Automation Systems

Existing laboratory automation systems primarily operate on rule-based logic and lack the ability to adapt dynamically to changing environments. While IoT-based systems have improved monitoring and control capabilities, most implementations still depend on predefined conditions rather than real-time decision-making. These systems are unable to differentiate between occupied and unoccupied zones within the same laboratory, resulting in inefficient energy utilization.

Furthermore, traditional systems do not provide fine-grained control over lighting. In most cases, the entire laboratory lighting is controlled as a single unit, even if only a small portion of the space is occupied. This lack of zonal awareness leads to excessive power consumption. Additionally, many systems do not offer real-time monitoring or user-friendly interfaces, limiting their usability and effectiveness in practical scenarios.

1.3 Role of Computer Vision and Intelligent Systems

Recent advancements in Computer Vision and Machine Learning have enabled the development of intelligent systems capable of understanding real-world environments. Object detection models such as YOLO (You Only Look Once) provide high accuracy and real-time performance in identifying human presence from video streams. Unlike traditional sensors, computer vision-based systems can detect both moving and stationary individuals, making them more reliable for occupancy detection.

The integration of such intelligent models with IoT devices enables the development of smart automation systems that can analyze real-time data and make informed decisions. Edge computing devices such as the Raspberry Pi 5 allow local processing of data, reducing latency and ensuring faster response times. This combination of computer vision and edge computing forms the foundation for next-generation smart laboratory systems.

1.4 Integration of IoT in Smart Automation

The Internet of Things (IoT) plays a crucial role in enabling connectivity and control within smart environments. IoT-based systems allow devices to communicate with each other and perform automated actions based on real-time data. In the context of laboratory automation, IoT enables centralized control of lighting systems, monitoring of energy usage, and seamless integration with user interfaces.

By connecting sensors, processing units, and control mechanisms, IoT facilitates the development of scalable and efficient automation systems. When combined with intelligent algorithms, IoT systems can move beyond simple automation and provide adaptive, data-driven solutions for energy management.

1.5 Towards Intelligent Zonal Lighting Control

To address the limitations of existing systems, there is a need for an intelligent solution that can provide zone-wise lighting control based on real-time occupancy. Instead of treating the entire laboratory as a single unit, dividing it into multiple zones allows more precise control of energy usage. By detecting human presence within specific zones, the system can activate only the required lighting units, thereby minimizing wastage.

This approach not only improves energy efficiency but also enhances system flexibility and scalability. It enables better utilization of resources and supports the development of smart infrastructure within educational institutions.

1.6 Proposed Smart Lab Automation System

The proposed system integrates Computer Vision, IoT, and Machine Learning to develop an intelligent Smart Lab Automation System. A camera continuously captures video input, which is processed using a YOLO-based model to detect human presence. The laboratory is divided into predefined zones, and each detected individual is mapped to a specific zone using spatial analysis techniques.

Based on occupancy information, the system dynamically controls lighting through GPIO pins connected to relay modules. A web-based interface is developed to provide real-time monitoring and manual control of the system. The use of edge computing ensures low latency and efficient operation without reliance on cloud infrastructure.

1.7 Vision and Impact

The primary objective of the proposed system is to reduce energy consumption while maintaining user comfort and operational efficiency. By enabling intelligent, real-time, and zonal control of lighting, the system contributes to sustainable energy management in educational environments. The solution is cost-effective, scalable, and suitable for deployment in various institutional settings.

This approach aligns with the global vision of developing smart and sustainable infrastructure by leveraging modern technologies. It represents a significant step towards transforming traditional laboratories into intelligent, energy-efficient systems capable of adapting to real-world conditions.

Chapter 2

Literature Survey/ Existing System

2. Survey/ Existing System

1.S. Y. Tupe, S. S. Kashid, S. S. Jamdade, S. S. Jadhav, and S. S. Ghatage – Computer Lab Automation System (2025)

The paper by S. Y. Tupe et al. presents an IoT-based computer lab automation system aimed at improving operational efficiency and reducing manual intervention. The system uses embedded controllers and sensors to automate electrical appliances such as lighting and computing systems. It provides centralized control for managing lab infrastructure and allows scheduling of operations to reduce energy wastage. The authors emphasize ease of implementation and cost-effectiveness. However, the system primarily operates on predefined rules and lacks real-time adaptability. It does not include intelligent decision-making or advanced detection mechanisms for occupancy. Additionally, the absence of zonal control results in inefficient energy utilization, as the entire lab is treated as a single unit. The study serves as a foundational approach to lab automation but highlights the need for integrating advanced technologies such as machine learning and computer vision for improved efficiency and accuracy.

2.Y. Nabae et al. – Digital Twin Framework for Smart Labs (2025)

Y. Nabae et al. introduce a digital twin framework for smart laboratories, where a virtual model replicates the physical lab environment. This framework enables real-time monitoring, simulation, and predictive analysis of lab operations. By integrating IoT sensors with data analytics, the system allows efficient resource management and performance optimization. The authors highlight the ability of digital twins to predict system failures and improve decision-making. However, the implementation requires high computational resources and complex integration, making it less feasible for small-scale applications. The study focuses more on monitoring and simulation rather than direct automation control. It lacks practical implementation of real-time lighting or occupancy-based control systems. Overall, the research demonstrates the potential of digital twins in smart environments but indicates the need for simpler, cost-effective solutions for real-world deployment.

3.K. Trikoupis et al. – Remote Laboratory for IoT Development and Learning Analytics (2024)

K. Trikoupis et al. propose a remote laboratory system that allows users to perform IoT experiments via internet-based platforms. The system integrates IoT devices with cloud computing to enable real-time interaction and data collection. It also includes learning analytics to assess user engagement and performance. This approach enhances accessibility and supports remote education. However, the system depends heavily on internet connectivity and cloud infrastructure, which may introduce latency issues. It focuses primarily on educational applications rather than energy efficiency or automation. Additionally, it lacks intelligent control of physical lab resources such as lighting or equipment. The study demonstrates the benefits of remote access but does not address real-time adaptive automation required in smart laboratories.

4. A. Mohankumar et al. – Wireless Smart Lighting System (2024)

A. Mohankumar et al. present a wireless smart lighting system designed to reduce energy consumption using motion sensors and wireless communication. The system automatically controls lighting based on detected movement, improving energy efficiency in indoor environments. The authors emphasize ease of installation and reduced wiring complexity. However, the reliance on motion sensors introduces limitations, such as failure to detect stationary individuals. The system also lacks zonal control, leading to inefficient energy usage when only partial occupancy exists. Furthermore, it does not include intelligent data processing

or predictive capabilities. The study highlights the benefits of smart lighting but suggests the need for more advanced technologies like computer vision for accurate occupancy detection.

5. A. N. Banagar et al. – IoT-Based Smart Laboratory System (2023)

A. N. Banagar et al. propose an IoT-based smart laboratory system for monitoring and controlling lab infrastructure. The system uses sensors to collect environmental data and automate device operations. It enables centralized control and improves efficiency in lab management. However, the system relies on rule-based automation and lacks intelligent decision-making. It does not adapt to real-time occupancy changes and lacks zonal control mechanisms. The absence of machine learning limits its ability to optimize energy usage dynamically. The study provides a scalable framework but highlights the need for intelligent automation.

6. A. M. Al Shahrani et al. – Machine Learning-Enabled Smart Industrial Automation Systems (2023)

A. M. Al Shahrani et al. explore the integration of machine learning with IoT sensors in industrial automation. The system uses ML algorithms to analyze sensor data and predict system behavior, improving efficiency and reducing downtime. While the study demonstrates the benefits of intelligent automation, it focuses on industrial applications rather than laboratory environments. It does not include occupancy detection or lighting control. The research highlights the importance of data-driven decision-making but requires adaptation for smart lab applications.

7. A. K. Rastogi et al. – IoT Enabled Machine Learning-Based Smart Systems (2023)

A. K. Rastogi et al. provide a comprehensive overview of IoT and machine learning-based smart systems. The paper discusses applications across various domains, including smart homes and industrial automation. It highlights the benefits of combining IoT data with ML algorithms for predictive analysis. However, the study is a general survey and does not focus specifically on laboratory automation. It lacks detailed implementation strategies for smart lab environments.

8. P. Sudhakaran et al. – Secured Smart Automation System for Computer Labs (2022)

P. Sudhakaran et al. propose a secure IoT-based automation system for computer labs. The system includes authentication mechanisms to prevent unauthorized access. While it improves security, it does not focus on energy efficiency or intelligent control. The automation is rule-based and lacks real-time adaptability. It also does not provide zonal control. The study emphasizes security but highlights the need for intelligent automation.

9. A. Raikwar et al. – IoT-Based Smart Classroom Automation System (2022)

A. Raikwar et al. present a smart classroom automation system using IoT. The system controls lighting and devices based on sensor inputs. It improves energy efficiency but relies on basic sensors. The system lacks intelligent processing and zonal control. It does not incorporate advanced detection methods like computer vision.

10. D. S. Jaykar et al. – Lab Automation System Using IoT (2022)

D. S. Jaykar et al. propose an IoT-based lab automation system for remote monitoring and control. The system improves convenience but relies on predefined rules. It lacks adaptability and intelligent decision-making. The absence of zonal control results in inefficient energy usage.

11. C. H. Ravali et al. – Smart Digital Laboratory Based on IoT (2022)

C. H. Ravali et al. present a smart digital lab system using IoT technologies. The system integrates sensors for automation and monitoring. However, it lacks advanced analytics and intelligent control mechanisms. It does not address zonal energy optimization.

12. F. Rahmanian et al. – Modular Autonomous Feedback-Loops (2022)

F. Rahmanian et al. introduce feedback-loop-based automation systems for adaptive decision-making. The system continuously monitors and adjusts operations based on feedback. While not specific to labs, the concept is relevant for smart automation systems.

Table 1. Research Paper

Year	Title	Domain	Algorithm	Dataset	Gap
2025	Computer Lab Automation System	Lab Energy Automation	Motion Detection	PIR Sensor Data	No cloud/mobile integration; basic detection only
2025	Digital Twin Framework for Smart Labs	Virtual Lab Simulation	Digital Twin + IoT	Simulated Lab Environments	Simulation focused; lacks real-world deployment and feedback loops
2024	Remote Laboratory for IoT Development and Learning Analytics	Remote IoT Education	Node-RED Workflow	LMS-integrated IoT logs	Limited tactile experience; latency issues; scalability challenges
2024	Wireless Smart Lighting System	Campus Infrastructure	Sensor-based LED Control	Wireless Sensor Logs	Centralised setup; poor modularity for decentralised labs
2023	IoT-Based Smart Laboratory System	Lab Automation	Rule-based Automation	Sensor Logs (Lights/Fans)	No ML integration; lacks adaptive intelligence
2023	ML-Enabled Smart Industrial Automation	Industrial IoT	SVM, Decision Trees	Industrial IoT Sensor Data	Focused on industry; lacks educational lab context

2023	IoT + ML Smart Systems Overview	Smart Systems	Hybrid ML Models	Aggregated IoT datasets	Broad overview; lacks implementation specifics for labs
2022	Secured Smart Automation System for Computer Labs	Lab Safety Automation	IoT Sensor Fusion	Motion, Fire, Intrusion Sensor Logs	Privacy concerns; cloud dependency; false alarms
2022	Smart Classroom Automation Using IoT	Educational Automation	Decision Trees, MQTT	Decision Trees, MQTT	Classroom focused; lacks lab specific energy optimization
2022	Lab Automation System using IoT	Lab Infrastructure Automation	Rule-Based + Wi-Fi Control	Sensor Logs (Lighting, HVAC)	No ML integration; lacks adaptive control and analytics
2022	A Robust Implementation Design Smart Digital Laboratory Based IoT	Smart Digital Lab	IoT + M2M Communication	Fan/Light Sensor Logs	No energy optimization; lacks intelligent feedback loops
2022	Modular Autonomous Feedback-Loops in Materials Science	Materials Lab Automation	Hierarchical ML + Orchestration	Experimental Lab Logs	Complex orchestration; limited generalizability to non-material labs

Chapter 3

Limitation of Existing System

3. Limitation of Existing System

Current laboratory automation and energy management systems suffer from several limitations that reduce their effectiveness:

- **Reactive Operation** – Most existing systems operate on predefined schedules or simple triggers rather than real-time conditions. They respond only after changes occur, rather than proactively optimizing energy usage based on continuous occupancy monitoring.
- **Dependence on Basic Sensors** – Many systems rely on sensors such as PIR (Passive Infrared), which detect only motion. These sensors fail to identify stationary individuals, leading to incorrect switching of lights and reduced system reliability.
- **Lack of Zonal Control** – Traditional automation systems treat the entire laboratory as a single unit. This results in all lights being turned ON even when only a small area is occupied, causing significant energy wastage.
- **Limited Intelligence and Adaptability** – Existing systems are largely rule-based and lack machine learning capabilities. They cannot learn from usage patterns or adapt dynamically to changing environments, limiting their efficiency in real-world scenarios.

- Inaccurate Occupancy Detection – Sensor-based systems provide limited spatial awareness and cannot accurately determine the number or position of occupants within a laboratory. This leads to inefficient resource utilization.
- High Energy Consumption – Due to the absence of intelligent control mechanisms, lights and devices often remain active unnecessarily, increasing overall power consumption and operational costs.
- Dependency on Cloud-Based Systems – Many modern systems rely on cloud computing for data processing, which introduces latency, increases dependency on internet connectivity, and raises data privacy concerns.
- Lack of Real-Time Processing – Existing solutions often fail to provide instant response to occupancy changes, resulting in delays in switching devices ON or OFF.
- Poor User Monitoring and Control Interfaces – Many systems lack intuitive dashboards or real-time monitoring capabilities, making it difficult for users to observe system performance and manually control operations when required.
- Limited Integration of Advanced Technologies – Most current solutions do not integrate computer vision or deep learning models for accurate human detection. This limits their ability to achieve precise and intelligent automation.

Chapter 4

Problem Statement and Objective

4.Problem Statement and Objective

4.1 Problem Statement

We are developing an intelligent, vision-based smart lighting system that automates lab lighting based on real-time occupancy. Instead of using fixed schedules or basic motion sensors, our system uses cameras and computer vision (YOLO) to accurately detect human presence. The system identifies which specific zones are occupied, ensuring that only required areas are illuminated. A Raspberry Pi (edge computing device) processes the video data locally, enabling fast and real-time decision-making without relying on cloud services. Based on the detected occupancy, a closed-loop control system dynamically turns lights ON or OFF and continuously updates decisions using feedback. The system sends control signals through GPIO to relay modules, which safely switch the LED lights for each zone. This enables zone-wise lighting control, reducing unnecessary energy usage. Overall, we are implementing a real-time, adaptive, and energy-efficient smart lighting solution that minimizes power wastage and improves automation in computer laboratories.

4.2 Objectives

- The system aims to develop a real-time occupancy detection mechanism using computer vision techniques, where a YOLO-based model processes live video streams to accurately identify human presence within the laboratory.
- The project focuses on implementing zonal lighting control by dividing the laboratory into multiple sections and dynamically controlling lights based on occupancy in each specific zone, thereby reducing unnecessary energy consumption.
- The system will utilize edge computing through a Raspberry Pi 5 to ensure fast processing, reduced latency, and minimal dependency on cloud infrastructure.
- A web-based monitoring and control interface will be developed to provide real-time visualization of system status, allowing users to monitor occupancy and manually control lighting when required.
- The system aims to optimize energy consumption by minimizing idle time of lighting systems and ensuring that only required areas are illuminated at any given time.
- The project also focuses on achieving high detection accuracy and fast response time to ensure reliable and efficient automation in real-world laboratory environments.

4.3 Scope

- The project enables real-time monitoring of laboratory occupancy using camera-based computer vision techniques for accurate detection of human presence.
- It provides intelligent zonal lighting control, ensuring that lights are activated only in occupied areas, leading to efficient energy utilization.
- The system supports real-time decision-making through edge computing, reducing latency and improving system responsiveness.
- A web-based dashboard is included to allow users to monitor system performance, visualize occupancy data, and manually override automated controls when necessary.

- The solution is scalable and can be extended to other environments such as classrooms, offices, and smart buildings for efficient energy management.
- The project integrates IoT, machine learning, and computer vision technologies to deliver a smart, adaptive, and energy-efficient automation system.

Chapter 5

Proposed System

Proposed System

5.1 Architecture of Proposed System

The proposed Smart Lab Automation System follows a layered and modular architecture that enables efficient real-time detection of human presence and intelligent control of lighting systems. The architecture integrates computer vision, IoT, edge computing, and web-based monitoring to achieve energy-efficient automation.

1. External Entities

The system interacts with multiple external entities that provide input and user interaction:

- **Camera :**

The camera acts as the primary input source, continuously capturing live video streams of the laboratory environment. These frames are used for detecting human presence using computer vision techniques.

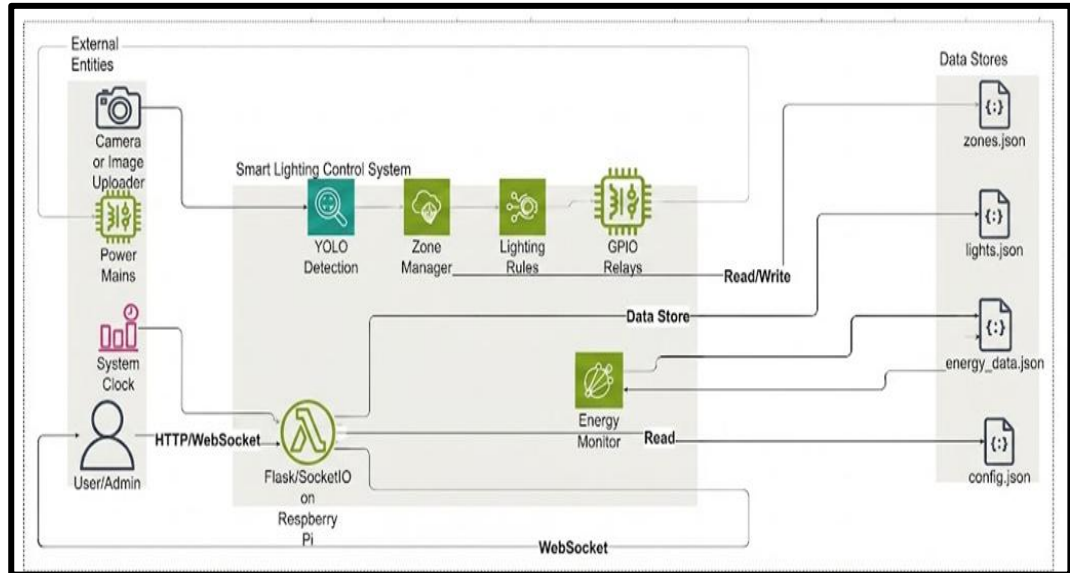


Figure 1: System Dataflow Architecture

- **Power Mains:**

This represents the electrical supply connected to the lighting system. The system controls power flow through relays to automate lighting.

- **System Clock:**

The clock provides time-based context for operations such as logging, monitoring energy usage, and applying timing rules (e.g., delay before switching OFF lights).

- **User/Admin:**

The user or administrator interacts with the system through a web interface using HTTP/WebSocket protocols. They can monitor system status and manually control lighting if required.

2. Core Smart Lighting Control System

This is the main processing unit where intelligent automation takes place. It consists of the following components:

- **YOLO Detection Module**

This module uses a deep learning-based YOLO (You Only Look Once) model to detect human presence in real-time video frames. It identifies individuals and generates bounding box coordinates, enabling accurate detection compared to traditional sensors.

- **Zone Manager**

The Zone Manager divides the laboratory into predefined zones. Based on the coordinates received from the YOLO model, it determines in which zone a person is present. This enables fine-grained control of lighting instead of treating the entire lab as a single unit.

- **Lighting Rules Engine**

This component applies logical rules to decide whether lights should be turned ON or OFF. The rules include:

- If a zone is occupied → Turn ON light
- If a zone is empty → Turn OFF light after delay
- Time-based conditions (optional)

This ensures efficient energy usage and avoids unnecessary switching.

GPIO Control (Relay Interface)

The GPIO module of the Raspberry Pi sends control signals to relay circuits. These relays physically switch the lights ON or OFF based on decisions made by the lighting rules engine. This acts as the bridge between software logic and hardware execution.

3. Backend and Communication Layer

Flask / Socket.IO Server

The backend is implemented using Flask with Socket.IO for real-time communication.

- HTTP: Used for standard requests such as loading the dashboard
- WebSocket: Enables real-time updates of system status

This layer connects the user interface with the core system and ensures seamless data exchange.

4. Energy Monitoring Module

The Energy Monitor tracks energy usage based on lighting activity.

- Reads data from system operations
- Calculates energy consumption
- Sends data for storage and visualization

This helps in evaluating system performance and measuring energy savings.

5. Data Storage Layer

The system uses lightweight JSON-based storage for managing data:

zones.json:

Stores configuration of different zones in the laboratory.

lights.json:

Maintains the current status (ON/OFF) of lights in each zone.

energy_data.json:

Stores energy consumption data for analysis and reporting.

config.json:

Contains system configuration parameters such as thresholds, delays, and rules.

These files are read/write accessible by the system, enabling dynamic updates and persistence.

6. Data Flow Summary

The overall working of the system can be summarized as follows:

- Camera captures real-time video
- YOLO model detects human presence
- Zone Manager identifies occupied zones

- Lighting Rules decide ON/OFF actions
- GPIO triggers relays to control lights
- Energy Monitor tracks usage
- Data is stored in JSON files

User/Admin monitors system via dashboard

5.2 Data Flow Diagram

Purpose:

The Data Flow Diagram (DFD) shows how data moves through the Smart Lab Automation System. It focuses on inputs such as video data, processing stages like human detection and decision-making, data storage, and outputs such as lighting control and dashboard updates. It provides a clear understanding of how the system processes real-time data to achieve intelligent automation.

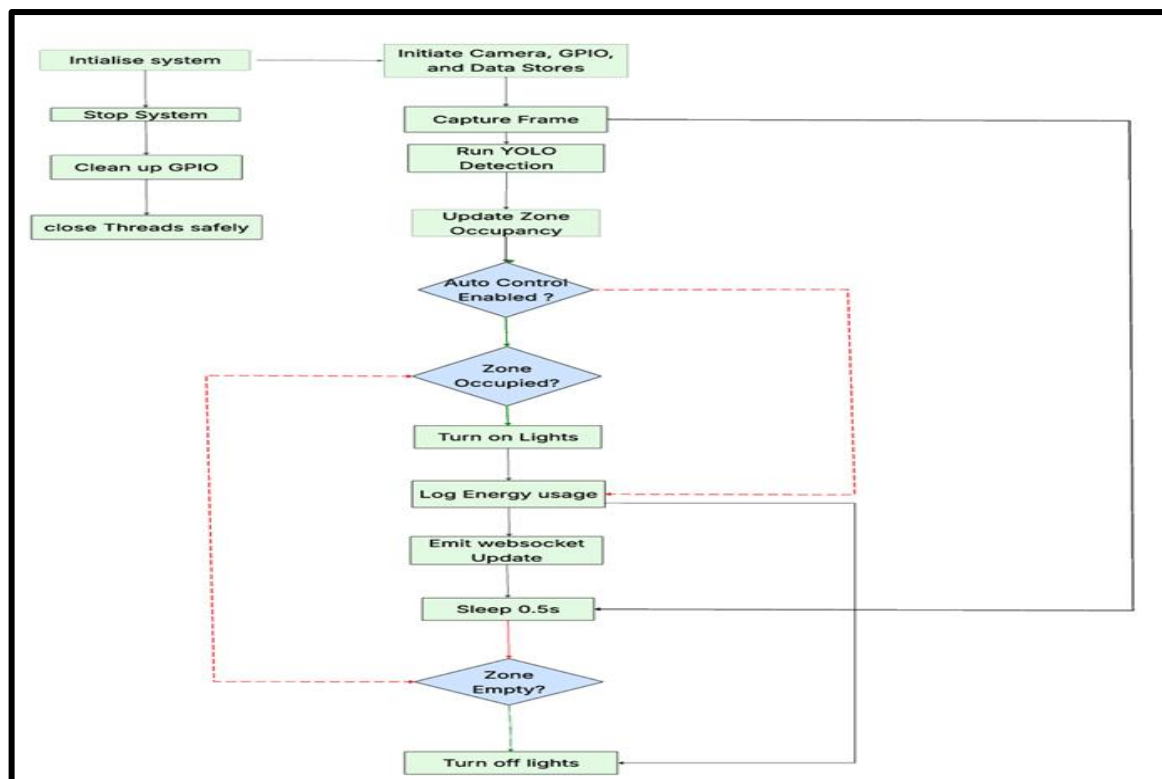


Figure 2 System Workflow

This diagram represents a workflow for an intelligent vision-based automation system that uses camera input and real-time processing to control devices such as lights based on occupancy detection.

The process begins with system initialization, where essential components like the camera, GPIO (General Purpose Input/Output), and data storage systems are activated. Once initialized, the system continuously captures frames from the camera for processing. Each frame is analyzed using a YOLO (You Only Look Once) object detection model, which identifies objects or people in the monitored area.

After detection, the system updates the zone occupancy status, determining whether a specific area is occupied or empty. This occupancy information is crucial for decision-making. The system then checks whether automatic control mode is enabled. If auto control is not enabled, the system continues monitoring without taking actions. If enabled, it proceeds to evaluate occupancy conditions.

When a zone is detected as occupied, the system automatically triggers actions such as turning on lights. It also logs energy usage to maintain a record of system activity and power consumption. Additionally, it sends real-time updates through a websocket connection, allowing remote monitoring of system status. The system then waits for a short interval (0.5 seconds) before repeating the process, ensuring continuous real-time operation.

If the system detects that the zone becomes empty, it proceeds to turn off the lights to conserve energy. This ensures efficient power usage by avoiding unnecessary energy consumption when no presence is detected.

The flow also includes safety and termination procedures. When the system is stopped, it performs cleanup operations such as releasing GPIO resources and safely closing all running threads. This prevents hardware conflicts and ensures stable shutdown.

Overall, the diagram illustrates a real-time smart automation system that combines computer vision, edge processing, and IoT-based control. It demonstrates how AI-powered detection can be integrated with hardware control to build an energy-efficient, intelligent environment management system.

5.3 Activity Diagram

Purpose:

The Activity Diagram represents the complete workflow of the Smart Lab System — from capturing input (camera) to controlling lights and generating outputs like energy data and dashboard updates. It shows decision-making and continuous processes.

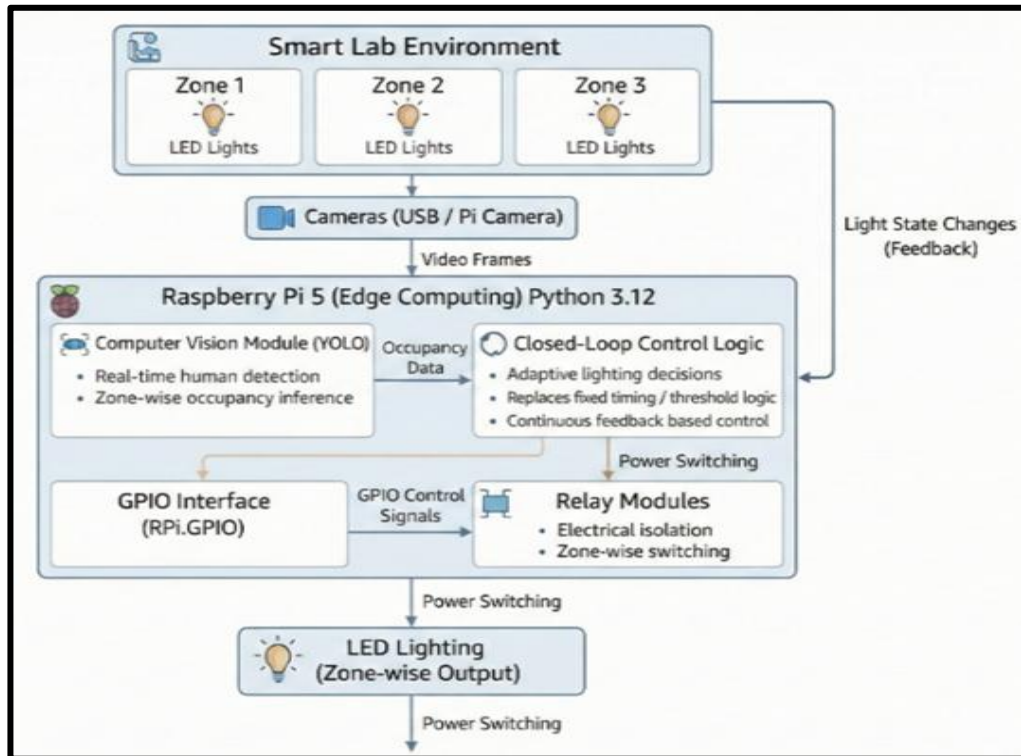


Figure 3 System Design

This diagram represents a Smart Lab Environment that uses edge computing, computer vision, and automated control systems to manage lighting efficiently across different zones.

At the top level, the lab is divided into three zones (Zone 1, Zone 2, and Zone 3), each equipped with LED lights. These zones are independently controlled, allowing the system to adjust lighting based on real-time conditions in each specific area rather than treating the entire lab as one unit. This zone-wise setup improves energy efficiency and provides more precise control.

Below the zones, cameras (USB or Pi Camera) continuously capture video frames from the lab environment. These cameras act as the input layer of the system, providing real-time visual data about human presence and movement in each zone.

The captured video frames are sent to a Raspberry Pi 5, which acts as the core processing unit using Python 3.12. This is an example of edge computing, meaning all processing happens locally on the device instead of relying on cloud servers. This reduces latency, improves response time, and ensures better privacy.

Inside the Raspberry Pi, there are two major functional modules:

Computer Vision Module (YOLO)

This module uses a real-time object detection algorithm (YOLO – You Only Look Once) to identify human presence in the video feed. It detects whether people are present in each zone and generates occupancy data. This enables the system to understand which areas are being used at any given moment.

Closed-Loop Control Logic

This module uses the occupancy data to make intelligent lighting decisions. Unlike simple systems that operate on fixed rules (like timers), this uses adaptive logic. It continuously adjusts lighting based on real-time feedback, such as whether people enter or leave a zone. This is called a closed-loop system because it constantly monitors output (light state) and adjusts input (control signals) accordingly.

The control logic sends signals to the GPIO Interface (RPi.GPIO), which is responsible for interacting with the physical hardware. These signals are passed to relay modules, which act as switches controlling the power supply to the LED lights. The relays provide electrical isolation and allow safe switching of high-voltage circuits using low-voltage signals from the Raspberry Pi.

The relay modules then control the LED lighting system, turning lights ON or OFF in each zone depending on occupancy. This ensures that lights are only used when needed, reducing energy consumption.

An important feature shown in the diagram is the feedback loop labeled “Light State Changes (Feedback).” This indicates that the system monitors the current lighting state and uses this information to refine future decisions. For example, if lights are already ON in a zone, the system won’t redundantly send another ON signal.

Overall, this system demonstrates an intelligent, automated lighting solution that combines computer vision, edge computing, and control systems to create an energy-efficient and responsive smart lab environment.

5.4 Key Components of the Proposed Framework

The proposed Smart Lab framework is designed using a modular and intelligent architecture to automate lighting control based on real-time occupancy detection. The system integrates computer vision, IoT, and edge computing to ensure efficient and adaptive operation.

The process begins with the input data source, where live video frames are captured using a camera installed in the laboratory. These frames act as the primary input for detecting human presence and monitoring activity within different zones of the lab.

Before applying detection algorithms, the captured frames undergo a data preprocessing stage. In this stage, image frames are resized, normalized, and converted into suitable formats using OpenCV. This preprocessing ensures that the input data is optimized for accurate and fast processing by the machine learning model.

After preprocessing, the frames are passed to the human detection model, which uses the YOLO (You Only Look Once) algorithm. This model identifies human presence in real time and generates bounding boxes along with confidence scores. Only detections above a predefined confidence threshold are considered to reduce false positives and improve reliability.

The detected outputs are then forwarded to the Zone Manager module, which maps each detected individual to predefined spatial zones in the laboratory. These zones are configured using coordinate boundaries stored in system files. Based on this mapping, the system continuously updates the occupancy status of each zone as either occupied or empty.

Following this, the control logic module (Lighting Rules) processes the occupancy data. This module applies decision-making rules such as turning lights ON when a zone is occupied and turning them OFF when the zone remains empty for a specific duration. This ensures energy-efficient operation without unnecessary switching.

The decisions are executed through the hardware control layer, where GPIO pins of the Raspberry Pi send signals to relay modules. These relay modules control the electrical circuits of the LED lights, enabling zone-wise switching of lighting systems.

Parallely, the system includes an energy monitoring module, which records lighting usage, duration, and power consumption. This data is stored in structured files and used to evaluate system performance and energy savings compared to traditional lighting systems.

The system also incorporates a web-based user interface, developed using Flask and SocketIO. This interface provides real-time visualization of zone occupancy, lighting status, and energy consumption. It also allows administrators to manually override automatic decisions when necessary.

Finally, all components work together in a closed-loop system, where continuous feedback from camera input and occupancy detection dynamically adjusts lighting conditions. This integrated framework ensures improved efficiency, reduced energy consumption, and scalable smart lab automation.

Chapter 6

Experimental Setup

6. Experimental Setup

6.1 Software Requirements

The Smart Lab Automation System is developed using a combination of modern software tools, programming languages, and machine learning frameworks to enable real-time processing, intelligent decision-making, and user interaction.

- Operating System:

- o Raspberry Pi OS (64-bit): Used as the primary operating system on the Raspberry Pi for running the core application, handling GPIO operations, and executing machine learning models.

- o Windows 10/11: Used as the development environment for coding, testing, and debugging the system components before deployment.

- o Python: The primary programming language used for backend development, machine learning model integration, GPIO control, and system logic implementation. It handles video processing, decision-making, and communication between modules.

- o HTML, CSS, JavaScript: Used for designing and developing the web-based user interface (dashboard), enabling real-time visualization and interaction with the system.

- Frontend Framework:

- o Basic Web Interface: Used to create a dynamic and responsive dashboard that allows administrators to monitor zone occupancy, control lighting, and view energy analytics in real time. It ensures a smooth and interactive user experience.

- Frameworks and Tools:

- o Flask: A lightweight Python web framework used to develop the backend server. It handles API requests, manages system logic, and connects the frontend with hardware components.

- o Flask-SocketIO: Enables real-time communication between the server and the web dashboard using WebSockets, ensuring instant updates of occupancy and lighting status.

- o OpenCV: A powerful computer vision library used for capturing and preprocessing video frames from the camera. It performs tasks such as resizing, normalization, and frame handling before passing data to the detection model.

- o TensorFlow Lite: A lightweight machine learning framework optimized for edge devices like Raspberry Pi. It is used to run the YOLO model efficiently with low latency.

- o YOLO (You Only Look Once): A real-time object detection algorithm used to detect human presence in the lab. It provides bounding boxes and confidence scores for accurate occupancy detection.

- Databases / Storage:

- o JSON Files: Used for lightweight and flexible data storage. Files such as zones.json, lights.json, and energy_data.json store zone configurations, lighting states, and energy logs.

- o SQLite (Optional): A local database that can be used for structured storage of historical data, improving scalability and analytics capabilities.

- Version Control:

- o Git & GitHub: Used for version control, code management, and collaboration. It helps track changes, maintain code history, and manage different versions of the project.

- Visualization Libraries:

- o Matplotlib: Used for plotting graphs related to energy consumption and system performance.

- o Plotly: Provides interactive visualizations for the dashboard, making it easier to analyze real-time data.

6.2 Hardware Requirements

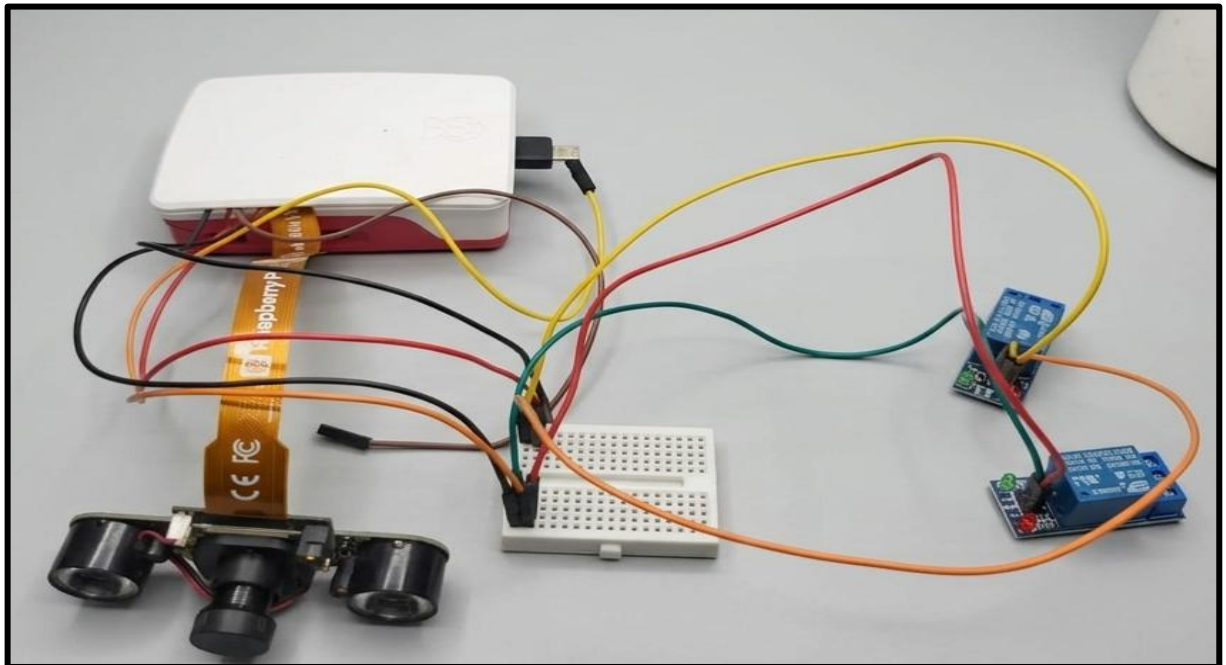


Figure 4 Hardware Setup

The hardware setup plays a crucial role in enabling real-time data acquisition, processing, and control of physical devices in the Smart Lab system.

- Processor

- o Raspberry Pi 5: Acts as the central processing unit of the system. It runs the backend server, processes video streams, executes the YOLO model, and controls hardware components through GPIO pins. Its high processing power allows efficient handling of multiple tasks simultaneously.

- Camera Module:

- o Raspberry Pi Camera Module V3: Used to capture live video feed of the laboratory environment. It provides high-resolution images and supports real-time video processing required for accurate human detection.

- Relay Module:

- o 4-Channel Relay Module: Used to control multiple lighting circuits independently. It acts as a switch between the Raspberry Pi and electrical devices, enabling automated ON/OFF control of lights.

- Lighting System:

- o LED Bulbs / LED Panels: Installed in different zones of the lab. These lights are controlled based on occupancy, ensuring energy-efficient operation.

- Power Supply:

- o 5V Power Adapter: Used to power the Raspberry Pi.

- o AC Power Supply: Used for lighting systems connected through relay modules.

- Additional Components:

- o Jumper wires for connections

- o Breadboard (for prototyping and testing)

Operating Environment

- IoT Device Environment:

The Raspberry Pi is connected to the camera and relay modules. It continuously captures video data, processes it locally, and controls lighting based on occupancy.

- Processing Environment:

The YOLO model runs on the Raspberry Pi using TensorFlow Lite. OpenCV handles frame preprocessing, ensuring efficient and accurate detection.

- Server Environment:

The Flask server runs on the Raspberry Pi, managing system logic, handling API requests, and enabling communication between frontend and backend.

- User Interface Environment:

A web-based dashboard allows users to monitor system status, view occupancy data, and control lighting manually if needed. It is accessible via browsers on desktops or mobile devices.

Development Environment

- Hardware Setup:

All hardware components (camera, relay module, Raspberry Pi) are connected and tested. GPIO pins are configured for controlling lighting circuits.

- Software Stack:

- Backend: Flask (Python)
- Frontend: HTML, CSS, JavaScript
- Database: JSON / SQLite

- IDE / Tools:

Machine Learning Libraries

- OpenCV: Used for image processing and frame handling
- TensorFlow Lite: Used for running ML models on edge devices
- YOLO: Used for real-time human detection

Chapter 7

Results and Discussion

7. Results and Discussion

1. System Performance and Data Acquisition

The proposed Smart Lab Automation System was successfully implemented using a camera-based vision system and IoT-enabled hardware to monitor real-time occupancy within the laboratory. The system continuously captured video frames using the camera module and processed them locally on the Raspberry Pi.

The integration of computer vision with edge computing ensured real-time data acquisition without dependency on cloud services, resulting in low latency and faster response times. The system demonstrated stable performance with minimal frame loss and consistent detection accuracy during operation.

Unlike traditional systems that rely on motion sensors or fixed schedules, this system continuously monitored the environment and dynamically adapted to occupancy changes. This continuous monitoring allowed precise identification of active and inactive zones, significantly improving system responsiveness and efficiency.

2. Machine Learning Model Evaluation

The YOLO-based human detection model performed efficiently in identifying human presence in real-time video streams. The model demonstrated high accuracy in detecting individuals across different lighting conditions and positions within the lab.

The model exhibited the following capabilities:

- Accurate real-time human detection
- Low false-positive rate due to confidence threshold filtering
- Fast processing suitable for edge devices like Raspberry Pi

The ability of the model to detect even partial human presence ensured reliable occupancy tracking. The lightweight implementation using TensorFlow Lite enabled smooth execution without overloading the system resources.

3. Zonal Control and Automation Performance

The Zone Manager module effectively mapped detected individuals to predefined spatial zones. Each zone operated independently, allowing fine-grained control of lighting.

Based on occupancy data:

- Lights were turned ON only in occupied zones
- Lights remained OFF in unoccupied areas
- Delay mechanism prevented frequent ON/OFF switching

This zonal approach significantly reduced unnecessary energy consumption compared to conventional systems where all lights remain ON regardless of occupancy.

The system maintained stable automation even during dynamic movement of users, ensuring accurate updates in occupancy states and lighting control.

4. Energy Efficiency and Analysis

One of the key outcomes of the system is the significant reduction in energy consumption. The energy monitoring module recorded lighting usage and calculated power consumption over time.

Observed results:

- Conventional system energy consumption: 24 Wh/day
- Smart system energy consumption: 9.5 Wh/day
- Energy savings: approximately 60%

This demonstrates that intelligent zonal control effectively minimizes energy wastage by eliminating idle lighting in unoccupied areas.

The system also maintained detailed logs of energy usage, enabling performance evaluation and comparison with traditional lighting systems.

5. User Interface and Accessibility

The web-based dashboard provided real-time visualization of system status, including:

- Zone occupancy status
- Lighting ON/OFF states
- Number of active lights
- Energy consumption data

The interface was designed to be simple and user-friendly, allowing easy monitoring and interaction. Administrators could also manually override automatic decisions when required.

Real-time updates using WebSocket communication ensured that users always had up-to-date information about the system.

6. Comparative Advantage Over Traditional Systems

The proposed Smart Lab system offers several advantages over traditional lighting systems:

- Energy Efficiency:

Traditional systems keep lights ON continuously, whereas the proposed system reduces energy consumption by up to 60%.

- Real-Time Adaptability:

Unlike fixed schedules or motion sensors, the system adapts dynamically based on actual occupancy.

- Higher Accuracy:

Computer vision-based detection is more accurate than PIR sensors, especially for stationary users.

- Automation with Control:

Provides both automatic operation and manual override options.

- Scalability:

Easily expandable to larger environments such as classrooms, offices, or smart buildings.

7. Overall Discussion

The Smart Lab Automation System successfully demonstrates the integration of computer vision, IoT, and edge computing to achieve intelligent and energy-efficient lighting control. The system not only improves operational efficiency but also contributes to sustainability by reducing energy wastage.

The results indicate that replacing conventional lighting systems with intelligent zonal control systems can significantly enhance energy management in academic and commercial environments.

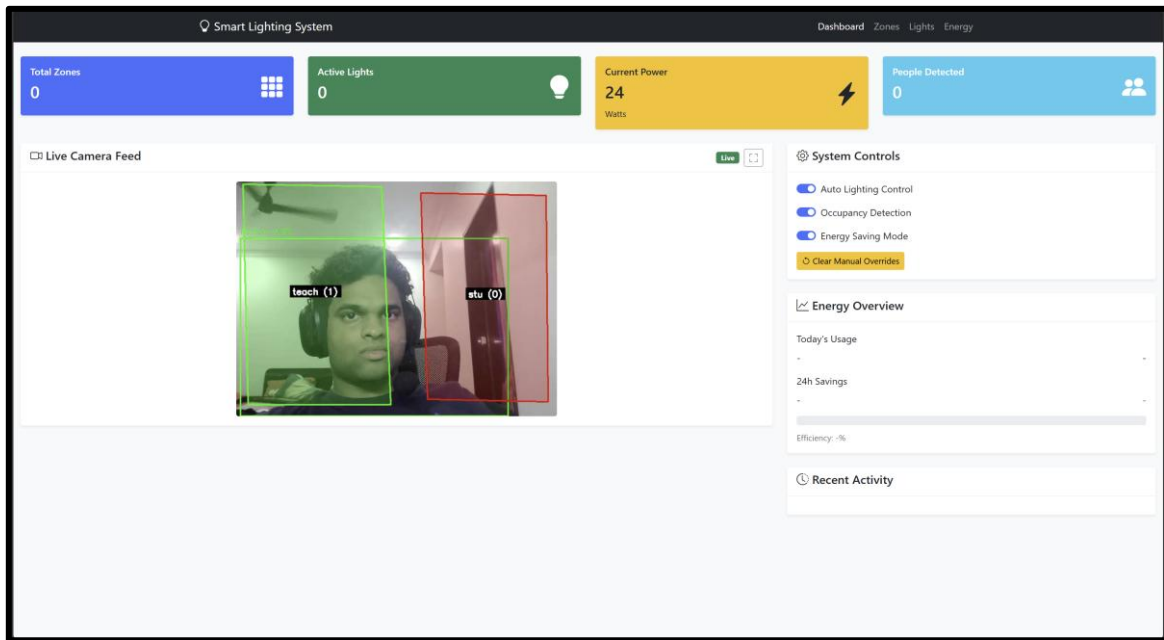


Figure 5 Live Camera feed

It uses a camera to monitor different zones and detect human presence through computer vision techniques. The system displays a live camera feed where individuals are identified and marked using bounding boxes. Based on occupancy detection, lights are automatically controlled, turning ON when a person is present and OFF when the area is empty, ensuring energy efficiency. The dashboard also shows key details such as total zones, active lights, power consumption, and people detected.

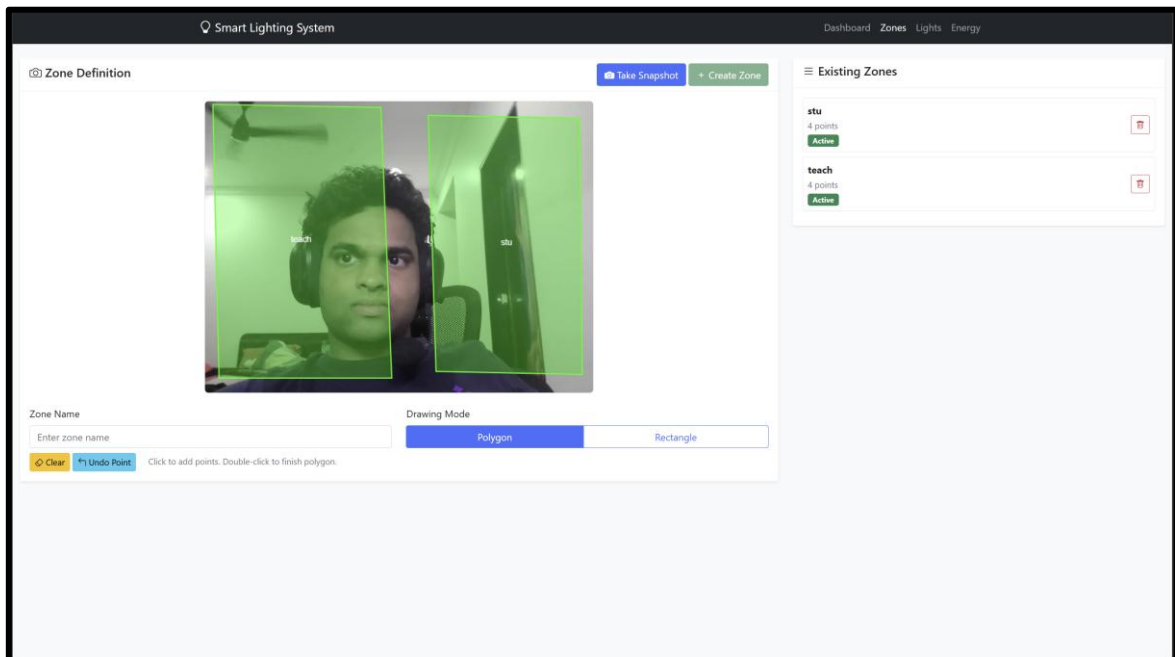


Figure 6 Zone Definition

In this interface, the user can define specific areas (zones) within the camera view by drawing shapes such as rectangles or polygons. These zones help the system monitor occupancy more accurately. Each zone is given a name and can be saved for further use. The live camera feed assists in placing zones correctly over real-world areas. Once defined, the system tracks movement within these zones and automatically controls lighting based on presence, improving energy efficiency and enabling smart automation.

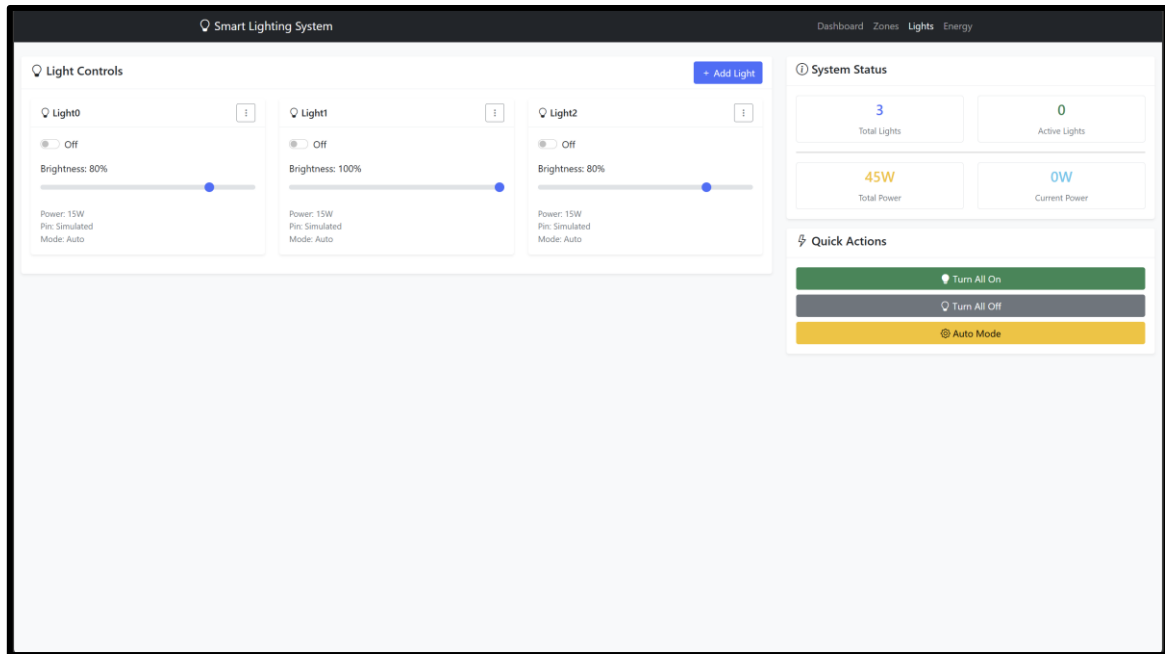


Figure 7 Light Control Page

Each light can be turned ON or OFF and its brightness can be adjusted using sliders. The system also displays power consumption and operating mode for each light. On the right side, system status provides information such as total lights, active lights, and total power usage. Quick action buttons like “Turn All On,” “Turn All Off,” and “Auto Mode” enable easy control. This interface ensures efficient energy management and user-friendly operation.

Table 2. Energy Consumption and Savings Comparison of Conventional and Smart Zonal Lighting Systems

Parameter	Conventional Lighting System	Smart Zonal Lighting System
Supply Voltage	5 V DC	5 V DC
Lighting Load	3 LEDs (1 per zone)	3 LEDs (1 per zone)
Control Method	Manual/ Always ON	Relay-based, ML-controlled
Total Daily Energy Consumption	24 Wh	9.5 Wh
Daily Energy Saving	0 Wh	14.5 Wh
Energy Consumption (10 Days)	0.72 kWh	0.285 kWh
Total Energy Saved (10 Days)	0 Wh	0.435 kWh
Percentage Energy Savings	0 Wh	≈ 60%

The table presents a comparison between the Conventional Lighting System and the Smart Zonal Lighting System based on energy usage and control mechanisms.

1. Supply Voltage

Both systems operate at 5V DC, ensuring that the comparison is made under identical electrical conditions.

2. Lighting Load

Each system uses 3 LEDs (one per zone). This keeps the hardware constant, so the difference in performance is due only to the control strategy.

3. Control Method

The conventional system uses a manual or always-on approach, where lights remain ON continuously.

The smart system uses relay-based control with machine learning, where lights operate based on real-time occupancy.

This difference in control is the key factor behind energy savings.

4. Total Daily Energy Consumption

Conventional system consumes 24 Wh per day

Smart system consumes 9.5 Wh per day

The smart system reduces unnecessary usage by turning lights ON only when required.

5. Daily Energy Saving

Conventional system shows no savings (0 Wh)

Smart system saves 14.5 Wh per day

This indicates significant daily energy conservation.

6. Energy Consumption (10 Days)

Conventional system: 0.72 kWh

Smart system: 0.285 kWh

Over time, the energy difference becomes more noticeable.

7. Total Energy Saved (10 Days)

The smart system saves 0.435 kWh over 10 days compared to the conventional system.

8. Percentage Energy Savings

The smart system achieves approximately 60% energy savings, showing a major improvement in efficiency.

Chapter 8

Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

The increasing demand for energy efficiency and sustainable infrastructure has made it essential to develop intelligent automation systems for managing resources effectively. Traditional lighting systems in laboratories and institutions often rely on manual control or fixed schedules, which leads to significant energy wastage due to lights remaining ON even when spaces are unoccupied. Such systems lack adaptability and fail to respond to real-time environmental conditions.

This project presented a Smart Lab Automation System that integrates computer vision, IoT, and edge computing to enable intelligent, occupancy-based lighting control. By using a camera module and a YOLO-based human detection model, the system continuously monitors the laboratory environment and identifies human presence in real time. The integration of a zone-based control mechanism ensures that lighting is activated only in occupied areas, thereby minimizing unnecessary energy consumption.

The use of a Raspberry Pi as an edge computing device allows all processing tasks, including detection and decision-making, to be performed locally. This reduces latency, enhances system responsiveness, and eliminates dependency on cloud-based processing. The relay-based control mechanism, connected through GPIO pins, enables efficient and reliable switching of lighting circuits.

The system also incorporates an energy monitoring module that records lighting usage and calculates energy consumption over time. Experimental results demonstrated a significant reduction in energy usage, achieving approximately 60% savings compared to conventional lighting systems. Additionally, the web-based dashboard provides real-time visualization, monitoring, and manual control, improving usability and system interaction.

Overall, the proposed system successfully demonstrates how modern technologies can be applied to create an intelligent, energy-efficient, and scalable automation solution. By combining real-time detection, zonal control, and IoT-based actuation, the system contributes to reducing energy wastage and promoting smart infrastructure in educational and commercial environments.

8.2 Future Work

Although the proposed Smart Lab Automation System achieves significant improvements in energy efficiency and automation, there are several opportunities for further enhancement and expansion.

Future work can focus on improving the accuracy and robustness of the human detection model by incorporating advanced deep learning techniques or optimizing the YOLO model for better performance under varying lighting conditions and occlusions. The system can also be extended to support multi-object tracking and behavior analysis, enabling more advanced automation features.

The integration of additional IoT sensors, such as ambient light sensors, motion sensors, and temperature sensors, can further enhance system intelligence. For example, combining occupancy detection with ambient light conditions can enable adaptive brightness control, improving both energy efficiency and user comfort.

Scalability can be improved by extending the system to larger environments such as entire buildings, offices, or campuses. This would involve centralized monitoring systems and cloud integration for managing multiple smart zones simultaneously. Cloud connectivity can also enable remote access, data storage, and advanced analytics.

Future developments can include the implementation of predictive analytics to forecast occupancy patterns and optimize energy usage in advance. Machine learning models can be trained on historical data to predict peak usage times and adjust system behavior accordingly.

Enhancing the user interface by developing a mobile application or more advanced web dashboard can improve accessibility and user interaction. Features such as notifications, usage reports, and system alerts can further enhance usability.

Finally, ensuring system security and data privacy will be critical for real-world deployment. Implementing secure communication protocols and access control mechanisms will protect system data and prevent unauthorized access.

Overall, future improvements can transform the proposed system into a more intelligent, scalable, and fully automated smart infrastructure solution suitable for real-world applications

References

- [1] S. Y. Tupe, S. S. Kashid, S. S. Jamdade, S. S. Jadhav, and S. S. Ghatage, "Computer lab automation system," *International Advanced Research Journal in Science, Engineering and Technology (IARJSET)*, vol. 12, Special Issue 1, Art. no. AITCON-14, 2025. <https://iarjset.com/paper/computer-lab-automation-system/>
- [2] Y. Nabae *et al.*, "Digital twin framework for smart labs," *Journal of Intelligent & Robotic Systems*, vol. 108, Art. no. 45, 2025. <https://link.springer.com/journal/10846>
- [3] K. Trikoupi *et al.*, "Remote laboratory for IoT development and learning analytics," *arXiv preprint*, 2024, arXiv:2402.05466. <https://arxiv.org/abs/2402.05466>
- [4] A. Mohankumar *et al.*, "Wireless smart lighting system," *Asian Journal of Applied Science and Technology (AJAST)*, vol. 10, Art. no. 8318, 2024. <https://www.ajast.in/>
- [5] A. N. Banagar *et al.*, "IoT-based smart laboratory system," *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 1, Art. no. IJERTV9IS010178, 2023. <https://www.ijert.org/iot-based-smart-laboratory-system>
- [6] A. M. Al Shahrani *et al.*, "Machine learning-enabled smart industrial automation systems using IoT sensors," *MDPI*, vol. 23, no. 1, Art. no. 324, 2023. <https://www.mdpi.com/1424-8220/23/1/324>
- [7] A. K. Rastogi *et al.*, "Internet of things enabled machine learning-based smart systems: A bird's eye view," *Engineering Proceedings*, vol. 59, Art. no. 36, 2023. <https://www.mdpi.com/2673-4591/59/1/36>
- [8] P. Sudhakaran *et al.*, "A secured smart automation system for computer labs in engineering colleges using IoT," *Computer Applications in Engineering Education*, vol. 30, no. 2, Art. no. CAE.22226, 2022. <https://onlinelibrary.wiley.com/doi/10.1002/cae.22226>
- [9] A. Raikwar *et al.*, "IoT-based smart classroom automation system," *International Journal of Advance Research and Innovative Ideas in Education (IJARIIE)*, vol. 9, no. 2, Art. no. 26185, 2022. <https://ijariie.com/FormDetails.aspx?MenuScriptId=26185>
- [10] D. S. Jaykar *et al.*, "Lab automation system using IoT," *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 9, no. 5, Art. no. JETIR2205986, 2022. <https://www.jetir.org/papers/JETIR2205986.pdf>

- [11] C. H. Ravali *et al.*, “A robust implementation design of smart digital laboratory based on IoT,” *JES Publications*, vol. 13, Issue 938, 2022.
<https://jespublication.com/volume13issue938/>
- [12] F. Rahmanian *et al.*, “Modular autonomous feedback-loops in materials science,” *Advanced Materials Interfaces*, vol. 9, Art. no. 2201234, 2022.
<https://onlinelibrary.wiley.com/doi/10.1002/admi.202201234>

Appendix

```
import base64
import json
import logging
import os
import threading
import time
from datetime import date, datetime
from typing import Dict, List

import cv2
import numpy as np
from flask import Flask, Response, jsonify, render_template, request
from flask_socketio import SocketIO, emit

# Import our custom modules
from src.camera_system import CameraSystem, encode_frame_to_jpeg,
resize_frame_for_web
from src.capture_logger import save_frame_and_log
from src.energy_monitor import EnergyMonitor
from src.lighting_system import LightingSystem
from src.zone_manager import ZoneManager

# Configure logging
logging.basicConfig(
    level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s"
)
logger = logging.getLogger(__name__)

class SmartLightingApp:
    """Main application class for the smart lighting system"""

    def __init__(self, config_file: str = "config.json"):
        # Load configuration
        self.config = self.load_config(config_file)

        # Initialize Flask app
        self.app = Flask(__name__)
        self.app.config["SECRET_KEY"] = "smart_lighting_secret_key_2024"
        # Use eventlet instead of threading for better compatibility
        self.socketio = SocketIO(
            self.app, cors_allowed_origins="*", async_mode="threading"
        )

        # Initialize system components
```

```

self.camera_system = None
self.zone_manager = None
self.lighting_system = None
self.energy_monitor = None

# Application state
self.running = False
self.main_loop_thread = None
self.last_update_time = time.time()

# Setup routes
self.setup_routes()
self.setup_socket_events()

def load_config(self, config_file: str) -> dict:
    """Load configuration from file"""
    try:
        if os.path.exists(config_file):
            with open(config_file, "r") as f:
                config = json.load(f)
        else:
            # Load from example config
            with open("config.example.json", "r") as f:
                config = json.load(f)

            # Save as main config
            with open(config_file, "w") as f:
                json.dump(config, f, indent=2)

            logger.info(f'Created config file from example: {config_file}')

    return config
    except Exception as e:
        logger.error(f'Failed to load config: {e}')
        raise

def initialize_systems(self):
    """Initialize all system components"""
    try:
        # Initialize camera system
        self.camera_system = CameraSystem(self.config)
        if not self.camera_system.start():
            raise RuntimeError("Failed to start camera system")

        # Initialize zone manager
        self.zone_manager = ZoneManager(self.config.get("zones", {}))

        # Initialize lighting system
        self.lighting_system = LightingSystem(self.config.get("lighting", {}))

```

```

# If there are no lights configured but zones define light_ids, register them
try:
    existing_lights = self.lighting_system.get_all_lights()
    if not existing_lights:
        # Preferred GPIO pins for lights (BCM)
        lighting_cfg = self.config.get("lighting", {})
        default_pins = lighting_cfg.get("gpio_pins", [17, 22, 27])
        pin_index = 0
        zones = self.zone_manager.get_all_zones()
        for zone_id, zone in zones.items():
            for lid in zone.light_ids:
                if self.lighting_system.get_light(lid) is None:
                    pin = default_pins[pin_index % len(default_pins)]
                    # Use zone name as light name
                    self.lighting_system.register_light(
                        light_id=lid,
                        name=zone.name or f"light_{pin}",
                        pin=pin,
                        power_watts=15.0,
                        zone_ids=[zone_id],
                    )
                    pin_index += 1
except Exception as e:
    logger.warning(f'Auto-registering lights from zones failed: {e}')

# Initialize energy monitor
self.energy_monitor = EnergyMonitor(self.config.get("energy", {}))

logger.info("All systems initialized successfully")

except Exception as e:
    logger.error(f'Failed to initialize systems: {e}')
    self.cleanup()
    raise

def start_main_loop(self):
    """Start the main lighting control loop in a separate thread"""
    if self.main_loop_thread is None or not self.main_loop_thread.is_alive():
        self.running = True
        self.main_loop_thread = threading.Thread(target=self.main_loop, daemon=True)
        self.main_loop_thread.start()
        logger.info("Main loop started successfully")
    else:
        logger.warning("Main loop is already running")

def setup_routes(self):
    """Setup Flask routes"""

    @self.app.route("/")
    def index():

```

```

        return render_template("index.html")

    @self.app.route("/api/health")
    def api_health():
        """Health check endpoint"""
        return jsonify(
            {
                "status": "ok",
                "systems_ready": self.all_systems_ready(),
                "main_loop_running": self.running
                and self.main_loop_thread
                and self.main_loop_thread.is_alive(),
                "timestamp": datetime.now().isoformat(),
            }
        )

    @self.app.route("/zones")
    def zones():
        return render_template("index_spa.html")

    @self.app.route("/lights")
    def lights():
        return render_template("index_spa.html")

    @self.app.route("/energy")
    def energy():
        return render_template("index_spa.html")

    @self.app.route("/debug")
    def debug():
        """Debug endpoint to test basic functionality"""
        try:
            zones_count = (
                len(self.zone_manager.get_all_zones()) if self.zone_manager else 0
            )
            lights_count = (
                len(self.lighting_system.get_all_lights())
                if self.lighting_system
                else 0
            )
            return f"Debug: {zones_count} zones, {lights_count} lights, Systems ready: {self.all_systems_ready()}"
        except Exception as e:
            return f"Debug error: {str(e)}"

    @self.app.route("/monitor")
    def monitor():
        """Real-time monitoring dashboard"""
        return render_template("index_spa.html")

```



```

# API Routes
@self.app.route("/api/status")
def api_status():
    """Get system status"""
    if not self.all_systems_ready():
        return jsonify({"error": "Systems not initialized"}), 503

    camera_status = self.camera_system.get_current_status()
    lighting_status = self.lighting_system.get_system_status()
    zone_stats = self.zone_manager.get_zone_statistics()

    # Get zone data and calculate summary stats
    all_zones = self.zone_manager.get_all_zones()
    active_zones = sum(
        1 for zone in all_zones.values() if zone.current_occupancy > 0
    )
    total_people = sum(zone.current_occupancy for zone in all_zones.values())

    return jsonify(
        {
            "camera": {
                "human_count": camera_status.get("human_count", 0),
                "has_frame": camera_status.get("has_frame", False),
            },
            "lights": {
                "total_lights": lighting_status.get("total_lights", 0),
                "lights_on": lighting_status.get("lights_on", 0),
                "lights_off": lighting_status.get("lights_off", 0),
            },
            "zones": {
                "total_zones": len(all_zones),
                "active_zones": active_zones,
                "total_people": total_people,
            },
            "energy": {
                "current_power": self.energy_monitor.get_current_power_consumption(),
                "currency": self.energy_monitor.currency,
            },
            "main_loop_running": self.running
            and self.main_loop_thread
            and self.main_loop_thread.is_alive(),
            "timestamp": datetime.now().isoformat(),
        }
    )

@self.app.route("/api/restart_main_loop", methods=["POST"])
def api_restart_main_loop():
    """Restart the main loop if it stopped"""
    try:
        if not (

```

```

        self.running
        and self.main_loop_thread
        and self.main_loop_thread.is_alive()
    ):
        logger.info("Restarting main loop...")
        self.start_main_loop()
        return jsonify({"status": "Main loop restarted", "success": True})
    else:
        return jsonify(
            {"status": "Main loop is already running", "success": True}
        )
except Exception as e:
    logger.error(f"Failed to restart main loop: {e}")
    return jsonify({"error": str(e), "success": False}), 500

@self.app.route("/api/camera/snapshot")
def api_camera_snapshot():
    """Get camera snapshot for zone definition"""
    if not self.camera_system:
        return jsonify({"error": "Camera not available"}), 503

    frame = self.camera_system.capture_snapshot()
    if frame is None:
        return jsonify({"error": "No frame available"}), 503

    # Resize for web display
    frame = resize_frame_for_web(frame, 800)

    # Set as reference frame
    self.zone_manager.set_reference_frame(frame)

    # Encode to base64
    _, buffer = cv2.imencode(".jpg", frame)
    frame_b64 = base64.b64encode(buffer).decode("utf-8")

    return jsonify(
        {
            "image": f"data:image/jpeg;base64,{frame_b64}",
            "width": frame.shape[1],
            "height": frame.shape[0],
            "timestamp": datetime.now().isoformat(),
        }
    )

# Note: image upload fallback removed - system relies on live camera feed

@self.app.route("/api/camera/stream")
def api_camera_stream():
    """Video streaming route for API endpoint"""
    return Response(

```

```

        self.generate_video_feed(),
        mimetype="multipart/x-mixed-replace; boundary=frame",
    )

    @self.app.route("/api/zones", methods=["GET", "POST"])
    def api_zones():
        """Zone management API"""
        if request.method == "GET":
            try:
                zones = self.zone_manager.get_all_zones()
                logger.info(f'API: Returning {len(zones)} zones')
                zone_dict = {
                    zone_id: zone.to_dict() for zone_id, zone in zones.items()
                }
                return jsonify({"zones": zone_dict})
            except Exception as e:
                logger.error(f'API: Error getting zones: {e}')
                return jsonify({"error": str(e)}), 500

        elif request.method == "POST":
            data = request.get_json() or {}
            try:
                # Basic validation
                if "name" not in data or not data["name"].strip():
                    return jsonify({"error": "Zone name is required"}), 400
                if (
                    "points" not in data
                    or not isinstance(data["points"], list)
                    or len(data["points"]) < 3
                ):
                    return (
                        jsonify({"error": "Zone must include at least 3 points"}),
                        400,
                    )

                zone_id = self.zone_manager.create_zone(
                    name=data["name"].strip(),
                    points=data["points"],
                    light_ids=data.get("light_ids", []),
                )

                # Return full zone data for immediate UI update
                zone = self.zone_manager.get_zone(zone_id)
                logger.info(
                    f'API: Created zone {zone_id} ({data["name"]}) with {len(data["points"])}
points"
                )
                return jsonify(
                    {"zone_id": zone_id, "zone": zone.to_dict(), "success": True}
                )

```

```

        except Exception as e:
            logger.error(f"API: Failed to create zone - {e}")
            return jsonify({"error": str(e)}), 400

    @self.app.route("/api/zones/<zone_id>", methods=["PUT", "DELETE"])
    def api_zone_detail(zone_id):
        """Individual zone management"""
        if request.method == "PUT":
            data = request.get_json()
            try:
                success = self.zone_manager.update_zone(
                    zone_id=zone_id,
                    name=data.get("name"),
                    points=data.get("points"),
                    light_ids=data.get("light_ids"),
                    enabled=data.get("enabled"),
                )
                return jsonify({"success": success})
            except Exception as e:
                return jsonify({"error": str(e)}), 400

        elif request.method == "DELETE":
            success = self.zone_manager.delete_zone(zone_id)
            return jsonify({"success": success})

    @self.app.route("/api/lights", methods=["GET", "POST"])
    def api_lights():
        """Light management API"""
        if request.method == "GET":
            try:
                lights = self.lighting_system.get_all_lights()
                logger.info(f"API: Returning {len(lights)} lights")
                lights_dict = {
                    light_id: light.to_dict() for light_id, light in lights.items()
                }
                return jsonify({"lights": lights_dict})
            except Exception as e:
                logger.error(f"API: Error getting lights: {e}")
                return jsonify({"error": str(e)}), 500

        elif request.method == "POST":
            data = request.get_json()
            try:
                light_id = self.lighting_system.create_light(
                    name=data["name"],
                    pin=data.get("pin"),
                    power_watts=data.get("power_watts", 15.0),
                    zone_ids=data.get("zone_ids", []),
                )
                return jsonify({"light_id": light_id, "success": True})

```

```

except Exception as e:
    return jsonify({"error": str(e)}), 400

@self.app.route("/api/lights/<light_id>", methods=["PUT", "DELETE"])
def api_light_detail(light_id):
    """Individual light management"""
    if request.method == "PUT":
        data = request.get_json()

        if "control" in data:
            # Light control (no brightness)
            desired_state = data["control"].get("state", False)
            light = self.lighting_system.get_light(light_id)
            previous_state = light.is_on if light else None
            success = self.lighting_system.control_light(
                light_id=light_id,
                state=desired_state,
                manual=data["control"].get("manual", False),
            )
            # Record energy event
            light = self.lighting_system.get_light(light_id)
            if light and success and previous_state != desired_state:
                self.energy_monitor.record_light_event(
                    light_id=light_id,
                    light_name=light.name,
                    event="on" if desired_state else "off",
                    power_watts=light.power_watts,
                )
            return jsonify({"success": success})
        else:
            # Light configuration update
            try:
                success = self.lighting_system.update_light(
                    light_id=light_id,
                    name=data.get("name"),
                    pin=data.get("pin"),
                    power_watts=data.get("power_watts"),
                    zone_ids=data.get("zone_ids"),
                )
                return jsonify({"success": success})
            except Exception as e:
                return jsonify({"error": str(e)}), 400

    elif request.method == "DELETE":
        success = self.lighting_system.delete_light(light_id)
        return jsonify({"success": success})

@self.app.route("/api/lights/auto-control", methods=["GET", "POST"])
def api_lights_auto_control():
    """Auto control management"""

```

```

if request.method == "GET":
    return jsonify(
        {"auto_control_enabled": self.lighting_system.auto_control_enabled}
    )

elif request.method == "POST":
    data = request.get_json()
    enabled = data.get("enabled", True)
    self.lighting_system.set_auto_control(enabled)
    if enabled:
        self.lighting_system.clear_all_manual_overrides()
    return jsonify(
        {
            "auto_control_enabled": self.lighting_system.auto_control_enabled,
            "success": True,
        }
    )

@self.app.route("/api/energy/daily")
def api_energy_daily():
    """Get daily energy usage"""
    target_date = request.args.get("date")
    if target_date:
        target_date = datetime.fromisoformat(target_date).date()

    return jsonify(self.energy_monitor.get_daily_usage(target_date))

@self.app.route("/api/energy/monthly")
def api_energy_monthly():
    """Get monthly energy usage"""
    year = request.args.get("year", type=int)
    month = request.args.get("month", type=int)

    return jsonify(self.energy_monitor.get_monthly_usage(year, month))

@self.app.route("/api/energy/weekly")
def api_energy_weekly():
    """Get weekly energy usage"""
    start_date = request.args.get("start_date")
    if start_date:
        start_date = datetime.fromisoformat(start_date).date()

    return jsonify(self.energy_monitor.get_weekly_usage(start_date))

@self.app.route("/api/energy/savings")
def api_energy_savings():
    """Get energy savings analysis"""
    days = request.args.get("days", 30, type=int)
    return jsonify(self.energy_monitor.calculate_energy_savings(days))

```

```

@self.app.route("/api/energy/patterns")
def api_energy_patterns():
    """Get usage patterns"""
    days = request.args.get("days", 7, type=int)
    return jsonify(self.energy_monitor.get_usage_patterns(days))

@self.app.route("/api/monitor/live")
def api_monitor_live():
    """Get live monitoring data for real-time dashboard"""
    if not self.all_systems_ready():
        return jsonify({"error": "Systems not initialized"}), 503

    # Get current detections and system status
    camera_status = self.camera_system.get_current_status()
    detections = camera_status.get("detections", [])

    # Get zone occupancy
    zone_occupancy = self.zone_manager.update_occupancy(detections)
    all_zones = self.zone_manager.get_all_zones()

    # Get lighting status
    all_lights = self.lighting_system.get_all_lights()

    # Compile comprehensive monitoring data
    monitoring_data = {
        "timestamp": datetime.now().isoformat(),
        "system_health": {
            "main_loop_running": self.running
            and self.main_loop_thread
            and self.main_loop_thread.is_alive(),
            "camera_active": camera_status.get("has_frame", False),
            "total_detections": len(detections),
            "systems_ready": self.all_systems_ready(),
        },
        "detections": {"count": len(detections), "people": detections},
        "zones": {
            "occupancy": zone_occupancy,
            "details": {
                zone_id: {
                    "name": zone.name,
                    "current_occupancy": zone.current_occupancy,
                    "last_occupied": (
                        zone.last_occupied.isoformat()
                        if zone.last_occupied
                        else None
                    ),
                },
                "enabled": zone.enabled,
            }
            for zone_id, zone in all_zones.items()
        },
    },

```

```

    },
    "lights": {
        "summary": {
            "total": len(all_lights),
            "on": sum(1 for light in all_lights.values() if light.is_on),
            "off": sum(
                1 for light in all_lights.values() if not light.is_on
            ),
        },
        "details": {
            light_id: {
                "name": light.name,
                "is_on": light.is_on,
                # 'brightness' removed
                "power_watts": light.power_watts,
                "manual_override": light.manual_override,
                "zone_ids": light.zone_ids,
            }
            for light_id, light in all_lights.items()
        },
    },
    "energy": {
        "current_power": self.energy_monitor.get_current_power_consumption(),
        "today_usage": self.energy_monitor.get_daily_usage(),
        "currency": self.energy_monitor.currency,
    },
}

return jsonify(monitored_data)

@self.app.route("/video_feed")
def video_feed():
    """Video streaming route"""
    return Response(
        self.generate_video_feed(),
        mimetype="multipart/x-mixed-replace; boundary=frame",
    )

def setup_socket_events(self):
    """Setup SocketIO event handlers"""

    @self.socketio.on("connect")
    def handle_connect():
        logger.info(f"Client connected: {request.sid}")
        # Start main loop when first client connects
        if not (
            self.running
            and self.main_loop_thread
            and self.main_loop_thread.is_alive()
        ):

```



```

        self.start_main_loop()
        emit("status", {"connected": True})

    @self.socketio.on("disconnect")
    def handle_disconnect():
        logger.info(f"Client disconnected: {request.sid}")

    @self.socketio.on("request_update")
    def handle_request_update():
        """Send real-time updates to client"""
        if self.all_systems_ready():
            self.emit_system_update()

def generate_video_feed(self):
    """Generate video feed for streaming"""
    while True:
        if self.camera_system:
            frame = self.camera_system.get_frame_with_detections()
            if frame is not None:
                # Draw zones on frame
                frame = self.zone_manager.draw_zones(frame, show_occupancy=True)

                # Resize for streaming
                frame = resize_frame_for_web(frame, 640)

                # Encode frame
                frame_bytes = encode_frame_to_jpeg(frame, quality=60)

                yield (
                    b"--frame\r\n"
                    b"Content-Type: image/jpeg\r\n\r\n" + frame_bytes + b"\r\n"
                )
            else:
                time.sleep(0.1)
        else:
            time.sleep(1)

def main_loop(self):
    """Main application loop for real-time lighting control"""
    logger.info("Entering main lighting control loop")
    iteration = 0

    while self.running:
        try:
            iteration += 1

            if iteration % 120 == 0: # Log every 60 seconds for system health
                logger.info(
                    f"Main loop iteration {iteration} - Systems running normally"
                )

```

```

# Check if we should exit due to system readiness
if not self.all_systems_ready():
    logger.warning("Systems not ready, exiting main loop")
    break

# Get current detections
detections = self.camera_system.get_current_status()["detections"]

# Update zone occupancy
zone_occupancy = self.zone_manager.update_occupancy(detections)

# Update lighting based on occupancy
lighting_results = self.lighting_system.update_zone_lighting(
    zone_occupancy
)

# Record energy events for changed lights (no logging)
for zone_id, controlled_lights in lighting_results.items():
    for light_id in controlled_lights:
        light = self.lighting_system.get_light(light_id)
        if light:
            self.energy_monitor.record_light_event(
                light_id=light_id,
                light_name=light.name,
                event="on" if light.is_on else "off",
                power_watts=light.power_watts,
                # brightness removed
            )
        # Save a capture showing the change and log it
        try:
            frame_for_log = self.camera_system.get_clean_frame()
            human_count = len(detections) if detections is not None else 0
            saved = save_frame_and_log(
                frame_for_log, light_id, zone_id, human_count
            )
            if saved:
                logger.debug(f"Logged capture for {light_id} -> {saved}")
        except Exception as e:
            logger.warning(f"Failed to save/log capture for {light_id}: {e}")

# Emit updates via SocketIO for web interface
current_time = time.time()
if (
    current_time - self.last_update_time > 1
): # Update every 1 second for web
    self.emit_system_update()
    self.last_update_time = current_time

time.sleep(0.5) # Main loop frequency

```

```

except Exception as e:
    import traceback

    logger.error(f"Error in main loop iteration {iteration}: {e}")
    logger.error(f"Exception type: {type(e).__name__}")
    logger.error(f"Full traceback:\n{traceback.format_exc()}")
    time.sleep(1)

logger.info("Main loop exited - self.running = " + str(self.running))

def emit_system_update(self):
    """Emit real-time system updates via SocketIO"""
    try:
        if not self.all_systems_ready():
            return

        # Get current camera status and detections
        camera_status = self.camera_system.get_current_status()
        detections = camera_status.get("detections", [])

        # Get zone data and calculate summary stats
        all_zones = self.zone_manager.get_all_zones()
        active_zones = sum(
            1 for zone in all_zones.values() if zone.current_occupancy > 0
        )
        total_people = sum(zone.current_occupancy for zone in all_zones.values())

        # Get lighting data and calculate summary stats
        all_lights = self.lighting_system.get_all_lights()
        lights_on = sum(1 for light in all_lights.values() if light.is_on)

        # Simple update data without problematic datetime formatting
        update_data = {
            "timestamp": datetime.now().isoformat(),
            "system_health": {
                "main_loop_running": self.running
                and self.main_loop_thread
                and self.main_loop_thread.is_alive(),
                "camera_active": camera_status.get("has_frame", False),
                "total_detections": len(detections),
                "systems_ready": self.all_systems_ready(),
            },
            "detections": {"count": len(detections), "people": detections},
            "zones": {
                "total_zones": len(all_zones),
                "active_zones": active_zones,
                "total_people": total_people,
                "zone_data": {
                    zone_id: {

```

```

        "name": zone.name,
        "occupancy": zone.current_occupancy,
        "last_occupied": (
            str(zone.last_occupied) if zone.last_occupied else None
        ),
        "enabled": zone.enabled,
    }
    for zone_id, zone in all_zones.items()
},
},
"lights": {
    "total_lights": len(all_lights),
    "lights_on": lights_on,
    "lights_off": len(all_lights) - lights_on,
    "light_data": {
        light_id: {
            "name": light.name,
            "is_on": light.is_on,
            # 'brightness' removed
            "power_watts": light.power_watts,
            "manual_override": light.manual_override,
            "zone_ids": light.zone_ids,
        }
        for light_id, light in all_lights.items()
    },
},
"camera": {
    "human_count": camera_status.get("human_count", 0),
    "has_frame": camera_status.get("has_frame", False),
    "detections": detections,
},
"energy": {
    "current_power": self.energy_monitor.get_current_power_consumption(),
    "daily_usage": self.energy_monitor.get_daily_usage(),
    "savings": self.energy_monitor.calculate_energy_savings(1),
    "currency": self.energy_monitor.currency,
},
}

# Emit general system update
self.socketio.emit("system_update", update_data)

# Emit specific monitoring data for real-time dashboard
self.socketio.emit(
    "monitor_update",
    {
        "detections": len(detections),
        "zones_occupied": active_zones,
        "lights_on": lights_on,
        "power_consumption": self.energy_monitor.get_current_power_consumption(),
    }
)

```

```

        "timestamp": datetime.now().isoformat(),
        "detailed_zones": {
            zone_id: {
                "name": zone.name,
                "occupancy": zone.current_occupancy,
                "status": (
                    "occupied" if zone.current_occupancy > 0 else "empty"
                ),
            }
            for zone_id, zone in all_zones.items()
        },
        "detailed_lights": {
            light_id: {
                "name": light.name,
                "status": "on" if light.is_on else "off",
                "power": light.power_watts if light.is_on else 0,
            }
            for light_id, light in all_lights.items()
        },
    },
)

```

```

except Exception as e:

```

```

    logger.error(f'Error emitting system update: {e}')

```

```

    import traceback

```

```

    logger.error(f'Full traceback: {traceback.format_exc()}')

```

```

def all_systems_ready(self) -> bool:

```

```

    """Check if all systems are initialized and ready"""

```

```

    return all(

```

```

        [

```

```

            self.camera_system is not None,

```

```

            self.zone_manager is not None,

```

```

            self.lighting_system is not None,

```

```

            self.energy_monitor is not None,

```

```

        ]

```

```

    )

```

```

def run(self, host: str = None, port: int = None, debug: bool = False):

```

```

    """Run the application"""

```

```

    try:

```

```

        # Initialize all systems

```

```

        self.initialize_systems()

```

```

        # Get server config

```

```

        server_config = self.config.get("server", {})

```

```

        host = host or server_config.get("host", "0.0.0.0")

```

```

        port = port or server_config.get("port", 5000)

```

```

        debug = debug or server_config.get("debug", False)

```

```

        logger.info(f"Starting Smart Lighting System on {host}:{port}")

        # Explicitly start the main loop before starting the server
        self.start_main_loop()

        # Run the Flask-SocketIO server
        logger.info(f"Starting SocketIO server on {host}:{port}")
        self.socketio.run(self.app, host=host, port=port, debug=debug)

    except KeyboardInterrupt:
        logger.info("Shutting down...")
    except Exception as e:
        import traceback

        logger.error(f"Application error: {e}")
        logger.error(f"Full traceback:\n{traceback.format_exc()}")
    finally:
        self.cleanup()

def cleanup(self):
    """Cleanup resources"""
    logger.info("Cleaning up resources...")
    self.running = False

    if self.camera_system:
        self.camera_system.stop()

    # Wait for main loop to finish
    if self.main_loop_thread and self.main_loop_thread.is_alive():
        self.main_loop_thread.join(timeout=5)

    if self.lighting_system:
        self.lighting_system.cleanup()

    if self.energy_monitor:
        self.energy_monitor.cleanup()

    logger.info("Cleanup completed")

# Create the main application instance
def create_app(config_file: str = "config.json"):
    """Create and configure the Flask application"""
    return SmartLightingApp(config_file)

if __name__ == "__main__":
    app = create_app()
    app.run()

```

Publication



International Conference on Emerging Technologies in Computing and Communication 2026 : Submission (176) has been edited.

1 message

Microsoft CMT <noreply@msr-cmt.org>
To: jay17112004patil@gmail.com

Sun, 15 Mar, 2026 at 8:16 pm

Hello,

The following submission has been edited.

Track Name: Communication and Industrial Automation

Paper ID: 176

Paper Title: Smart labs zonal control using machine learning

Abstract:

Modern laboratory in universities nowadays rely heavily on digital infrastructure which consume high energy and causes a lot of waste in energy consumption due to irregular occupancy. Conventional automation system techniques are made to avoid this energy waste crisis such as using basic motion sensors , using systems on a fixed schedule but this leads to often failing to adapt real usage patterns within laboratory spaces. This paper proposes a Smart lab automation system that uses IoT(Internet of Things) hardware with machine learning with computer vision to enable zone wise intelligent lighting control. The proposed system uses a Raspberry pi 5 as an edge computing unit to control the automation system by processing real time video streams, where human presence is detected using a YOLO based model. When a human presence is detected the lights in the specific occupied zone are dynamically controlled by the GPIO pins on the raspberry pi 5 by relay modules . This ensures that only required areas are illuminated. A web based monitoring and control system is designed using Flash and Socket IO which allows users to observe system status and manually control automated system decision whenever necessary. The experimental evaluation of the proposed system shows an energy reduction ranging from 35% to 70% with good response time and operational accuracy compared to traditional systems and conventional sensor based automation systems. Keywords- Smart Lab Automation, IoT, Raspberry Pi, YOLO, Computer Vision, Energy Optimization, Zonal Control.

Created on: Sun, 15 Mar 2026 14:38:17 GMT

Last Modified: Sun, 15 Mar 2026 14:46:07 GMT

Authors:

- jay17112004patil@gmail.com (Primary)
- karanvethody22@apsit.edu.in
- vedantvethekar87@apsit.edu.in
- shdalvi@apsit.edu.in
- svnair@apsit.edu.in

Primary Subject Area: Internet of things and smart cities

Secondary Subject Areas: Not Entered

Submission Files:

smart lab zonal control using machine learning.pdf (875 Kb, Sun, 15 Mar 2026 14:38:06 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

Acknowledgements

We make a move to offer our genuine thanks to **Prof. Shraddha Dalvi** for his important direction in this task. We are especially appreciative for his untiring help with this project as well as for empowering us in taking out the mistake and kept our advancement on plan. We are likewise thankful for the co-activity and significant help delivered by **Prof. Sayali Badhan** as Project Co-ordinator & **Dr. Jaya Gupta** as Head of the Department. We might want to offer our unique gratitude to the next instructing and non-showing staff of the AIML Department for their valuable help in the finish of this task. At long last, we are enormously obliged to the administration of our school and express our appreciation to the Principal **Dr. Uttam Kolekar** for organizing and giving essential office in finishing this venture work. We wish to recognize the assistance gave by our kindred schoolmates to aiding the assets while finishing this undertaking. We truly thank all other people who have assisted us in this venture with working.

Karan Vethody 22106023

Vedant Vethekar 22106090

Jay Patil 22106020

Date: